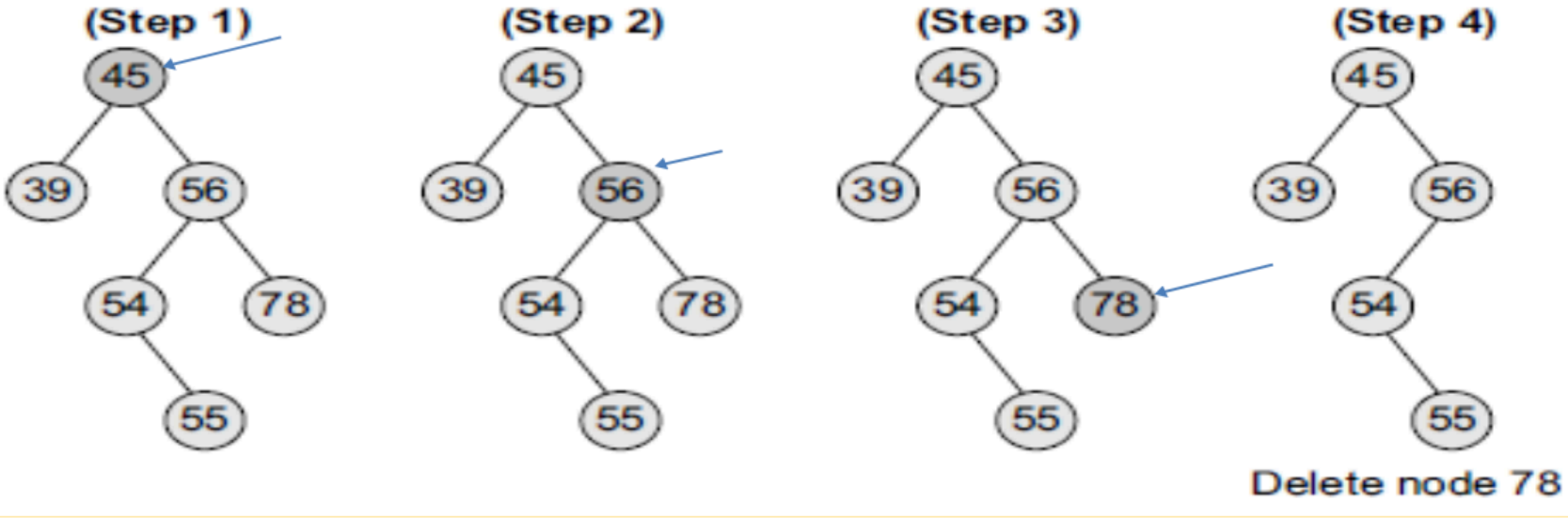
RB Tree Deletion

Node Deletion
in
Binary Search Tree

Deletion from Binary Search Tree

Case 1: Deleting a Node that has No Children

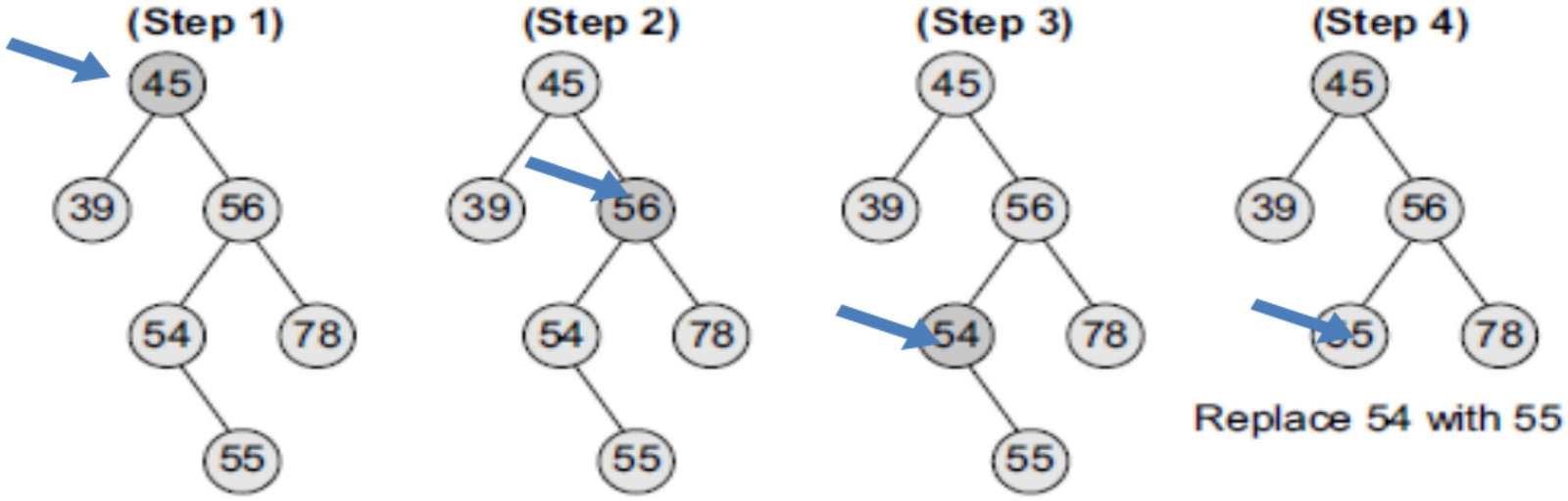


If we have to delete node 78,
we can simply remove this node without any issue.
This is the simplest case of deletion

Case 2: Deleting a Node with One Child

the **node's child is set as the child of the node's parent**.
In other words, replace the node with its child.

Deletion of **node 54**

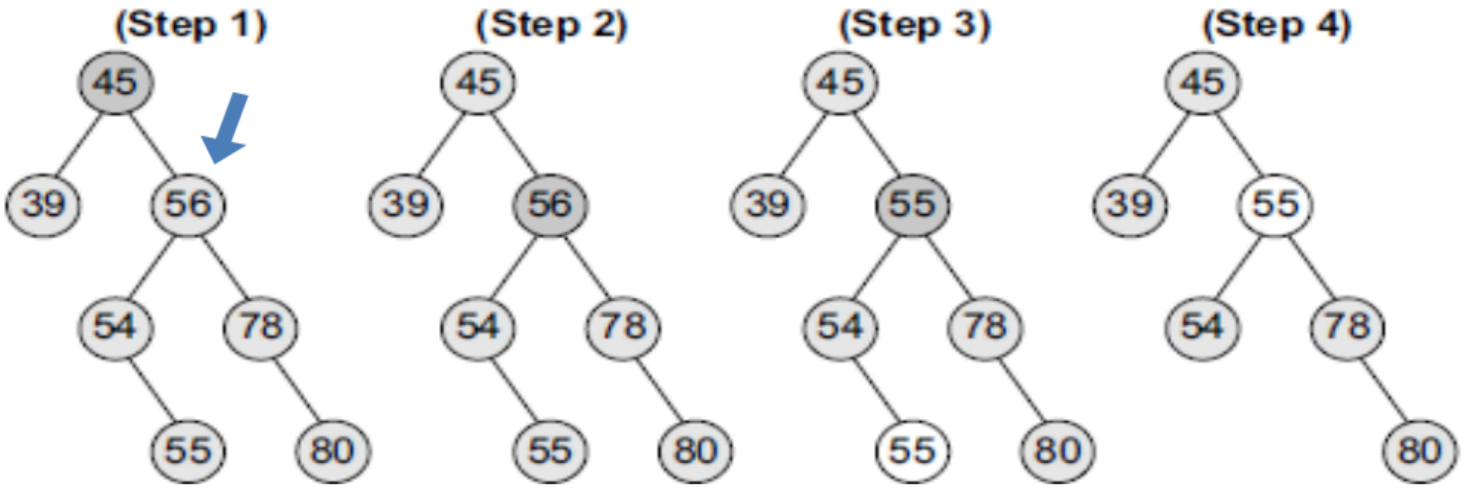


- if the node is the left child of its parent, the node's child becomes the left child of the node's parent.
- if the node is the right child of its parent, the node's child becomes the right child of the node's parent.

Case 3: Deleting a Node with Two Children

Replace the node's value with its
in-order predecessor (largest value in the left sub-tree) or
in-order successor (smallest value in the right sub-tree).

deletion of node with **value 56**.



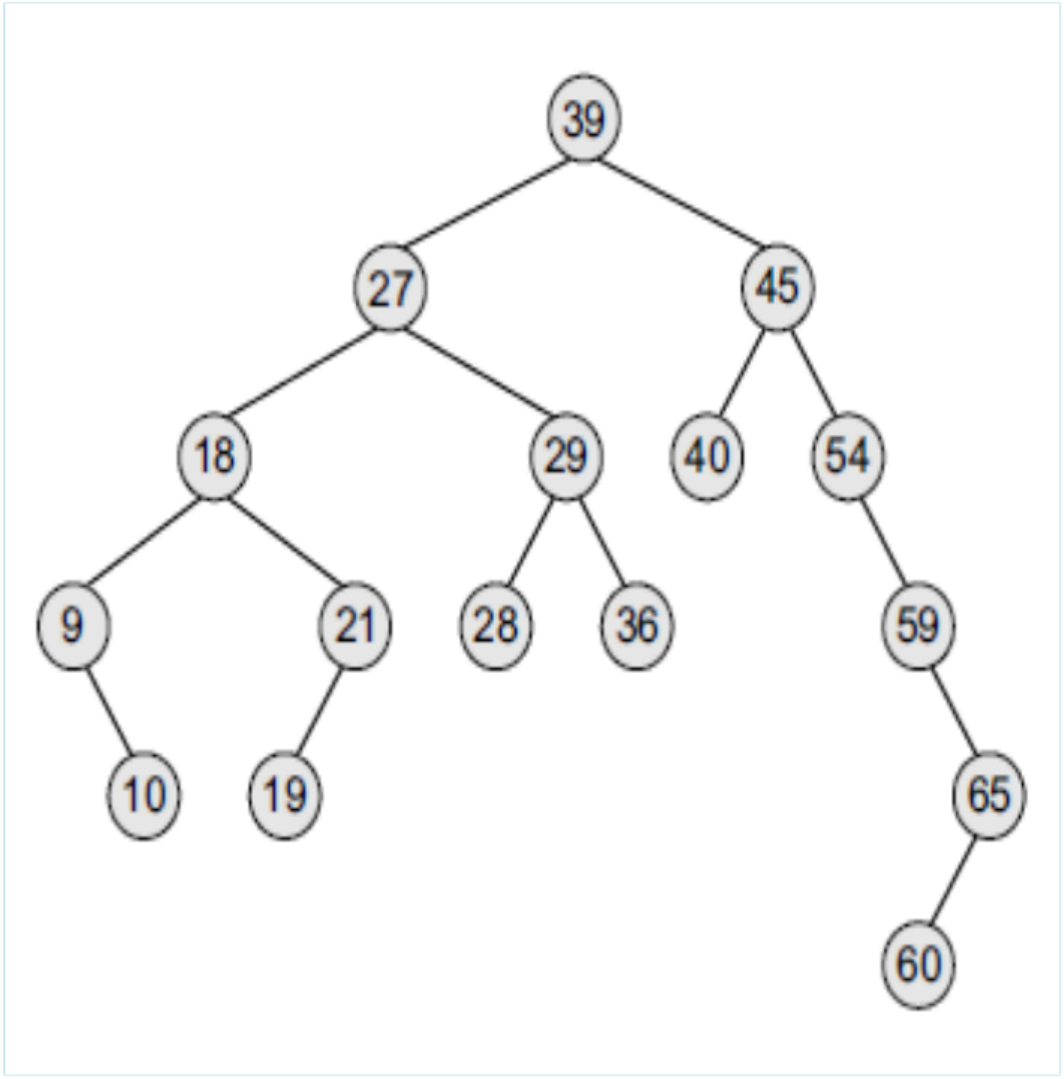
Using in-order predecessor in the above case.

Algorithm for Deletion

```
Delete (TREE, VAL)

Step 1: IF TREE = NULL
          Write "VAL not found in the tree"
        ELSE IF VAL < TREE -> DATA
          Delete(TREE->LEFT, VAL)
        ELSE IF VAL > TREE -> DATA
          Delete(TREE -> RIGHT, VAL)
        ELSE IF TREE -> LEFT AND TREE -> RIGHT
          SET TEMP = findLargestNode(TREE -> LEFT)
          SET TREE -> DATA = TEMP -> DATA
          Delete(TREE -> LEFT, TEMP -> DATA)
        ELSE
          SET TEMP = TREE
          IF TREE -> LEFT = NULL AND TREE -> RIGHT = NULL
               SET TREE = NULL
          ELSE IF TREE -> LEFT != NULL
                SET TREE = TREE -> LEFT
          ELSE
                SET TREE = TREE -> RIGHT
          [END OF IF]
          FREE TEMP
        [END OF IF]
Step 2: END
```

Example- BST Deletion



RB Tree Deletion

Step 1 - Perform BST deletion

Step 2

Case 1-If node to be deleted is **RED** => just delete it [**Terminal**]

Case 2-If node to be deleted is **BLACK** but has **RED** child

=> Replace node with child and color it with BLACK

Or if the Node to be deleted is **BLACK** & has **RED** parent => swap the color of Parent & Sibling (terminal)

Case 3(1) If root node is DB, => remove DB [**Terminal**]

Case 3(2) - If **DB** sibling is **RED** (also has **BLACK** parent)

➜ swap colors of PARENT and its SIBLING

➜ rotate PARENT in DB direction

➜ reapply cases

Case 3(3) - If **DB**'s sibling is **BLACK** and both of its childr
are **BLACK**

➜Remove DB (node)

= => Add extra black to its parent

=====> if parent is RED, then make it BLACK

======> if parent is BLACK, then make it double BLACK

☒ Make sibling RED

➜ If stillDB exists, apply other cases.

Case 3(4) - DB's sibling is **BLACK**, check sibling child's who is far from DB is **BLACK** but near child to DB is **RED**

➜Swap color of DB's sibling & siblings child who is near to DB

➜Rotate sibling in opposite direction to DB

➜Apply case 6

Case 3(5) - DB's sibling is **BLACK** and far child is **RED** [**Terminal**]

➜Swap color of PARENT & SIBLING

➜Rotate PARENT in DB's direction

➜Remove DB

➜Change color of RED child(far) to BLACK

[if node to be deleted is black , it will be replaced by Double Black DB]

RB tree Deletion

Construct a tree as follows

49, 28, 58, 14, 30, 52, 67, 63, 75, 79

Perform the following deletion

52, 28, 79, 75, 49, 30, 14

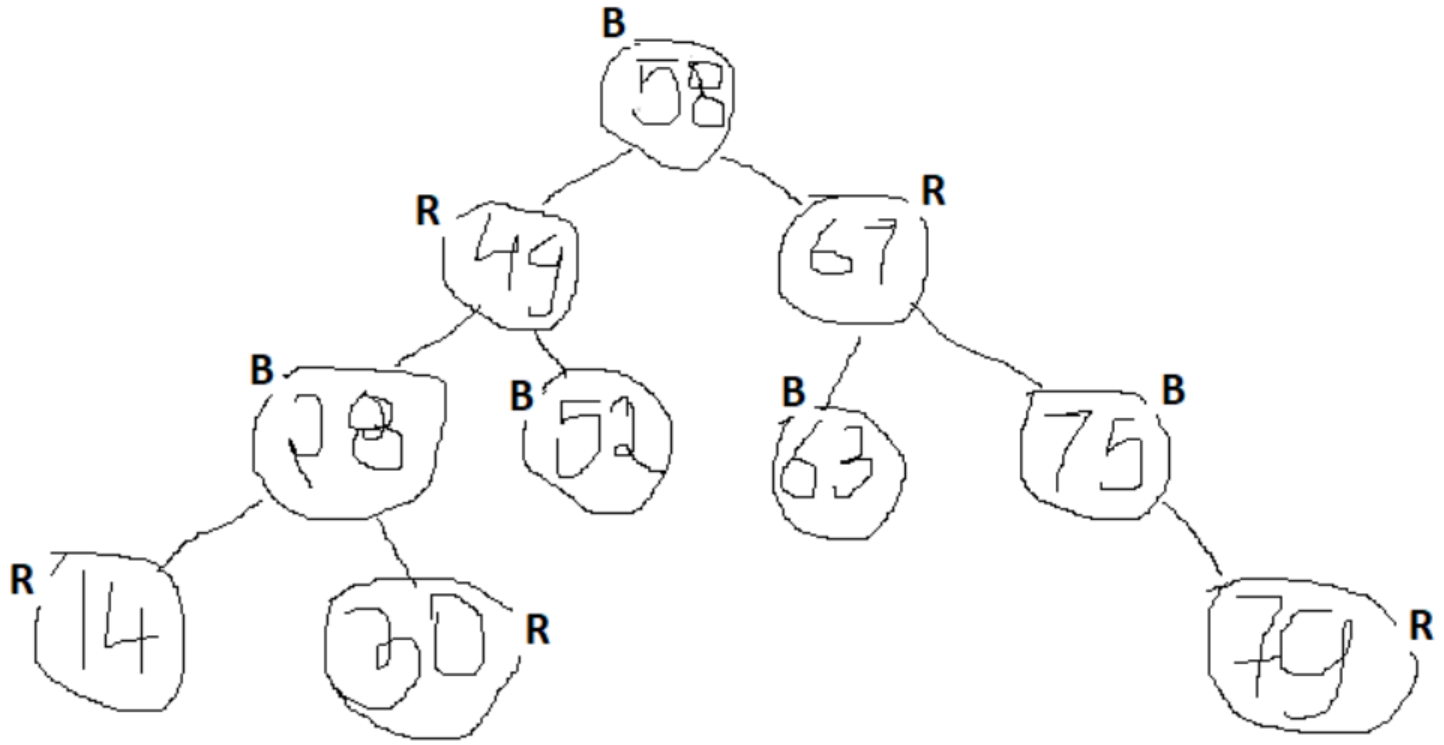
RB tree Deletion

Construct a tree as follows

49, 28, 58, 14, 30, 52, 67, 63, 75, 79

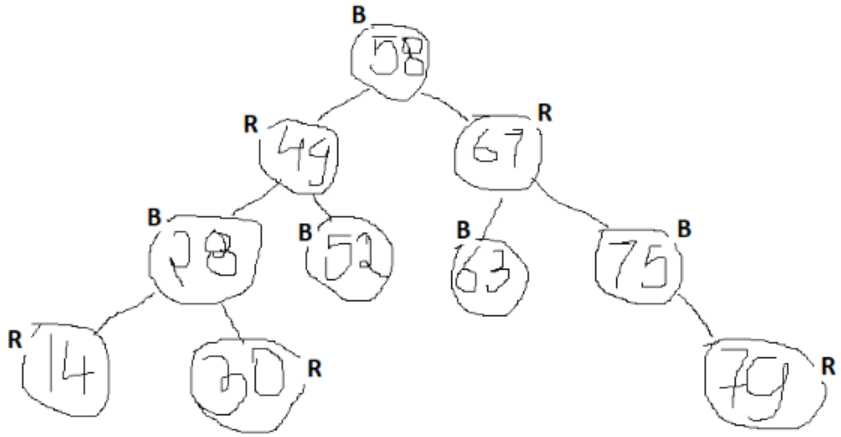
Perform the following deletion

52, 28, 79, 75, 49, 30, 14



Perform the following deletion

52, 28, 79, 75, 49, 30, 14



Step 1 - Perform BST deletion

Step 2

Case 1-If node to be deleted is **RED** => just delete it [**Terminal**]

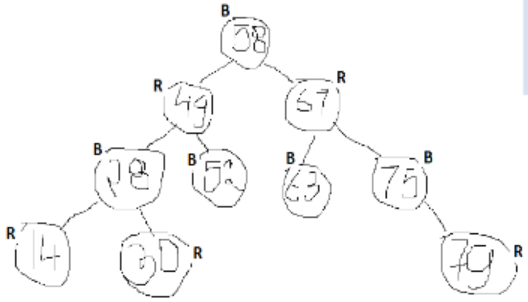
Case 2-If node to be deleted is **BLACK** but has **RED** child

=> Replace node with child and color it with BLACK

Or if the Node to be deleted is **BLACK** & has **RED** parent => swap the color of Parent & Sibling (terminal)

RB Tree Deletion

52, 28, 79, 75, 49, 30, 14



Case 3(1) If root node is DB, => remove DB [**Terminal**]

Case 3(2) - If **DB** sibling is **RED** (also has **BLACK** parent)

➔ swap colors of PARENT and its SIBLING

➔ rotate PARENT in DB direction

➔ reapply cases

Case 3(3) - If **DB**'s sibling is **BLACK** and both of its children are **BLACK**

➔Remove DB

= => Add extra black to its parent

=====> if parent is RED, then make it BLACK

======> if parent is BLACK, then make it double BLACK

☒ Make sibling RED

➔ If stillDB exists, apply other cases.

Case 3(4) - DB's sibling is **BLACK**, check sibling child's who is far from DB is **BLACK**

but near child to DB is **RED**

➔Swap color of DB's sibling & siblings child who is near to DB

➔Rotate sibling in opposite direction to DB

➔Apply case 3(5)

Case 3(5) - DB's sibling is **BLACK** and far child is **RED** [**Terminal**]

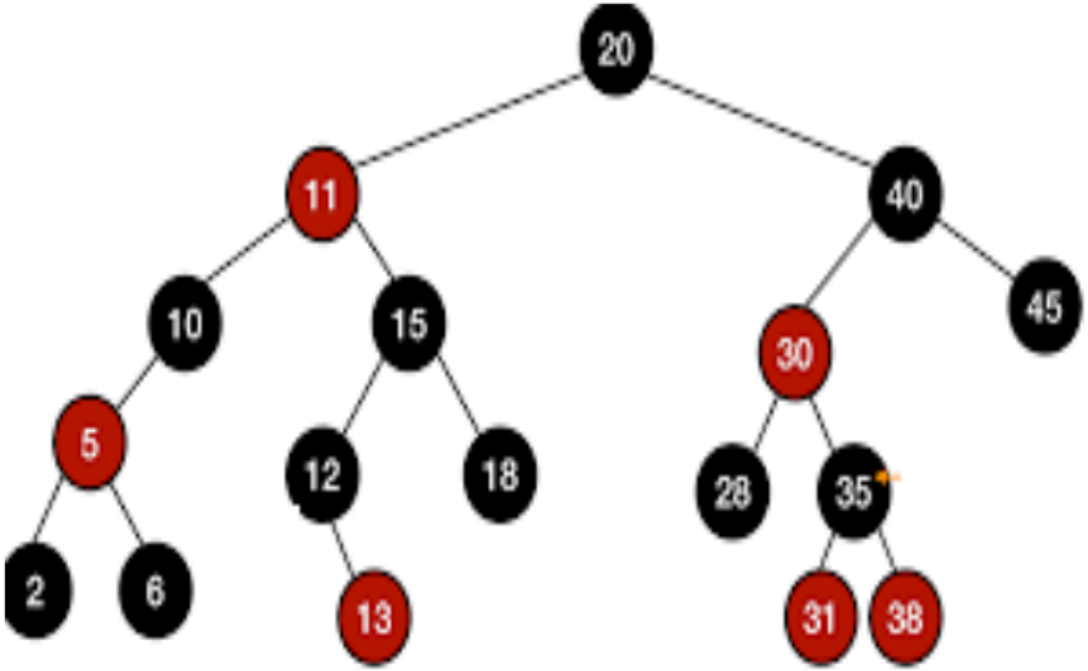
➔Swap color of PARENT & SIBLING

➔Rotate PARENT in DB's direction

➔Remove DB

➔Change color of RED child(far) to BLACK

Delete 30, 45, 10, 12, 35



Step 1 - Perform BST deletion

Step 2

Case 1-If node to be deleted is **RED** => just delete it **[Terminal]**

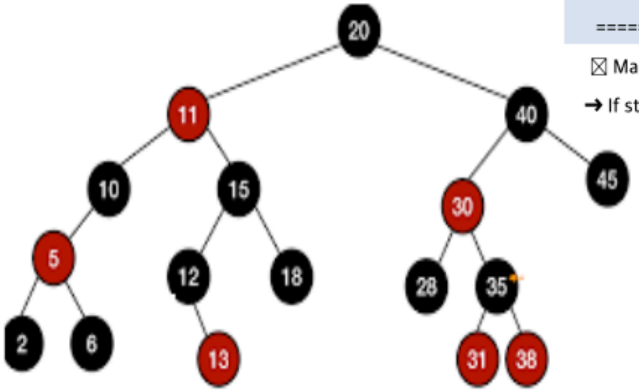
Case 2-If node to be deleted is **BLACK** but has **RED** child

=> Replace node with child and color it with BLACK

Or if the Node to be deleted is **BLACK** & has **RED** parent => swap the color of Parent & Sibling (terminal)

RB Tree Deletion

Delete 30, 45, 10, 12, 35



Case 3(1) If root node is DB, => remove DB **[Terminal]**

Case 3(2) - If **DB** sibling is **RED** (also has **BLACK** parent)

→ swap colors of PARENT and its SIBLING

→ rotate PARENT in DB direction

→ reapply cases

Case 3(3) - If **DB**'s sibling is **BLACK** and both of its children are **BLACK**

→Remove DB

= => Add extra black to its parent

=====> if parent is RED, then make it BLACK

======> if parent is BLACK, then make it double BLACK

☒ Make sibling RED

→ If stillDB exists, apply other cases.

Case 3(4) - DB's sibling is **BLACK**, check sibling child's who is far from DB is **BLACK**

but near child to DB is **RED**

→Swap color of DB's sibling & siblings child who is near to DB

→Rotate sibling in opposite direction to DB

→Apply case 3(5)

Case 3(5) - DB's sibling is **BLACK** and far child is **RED** **[Terminal]**

→Swap color of PARENT & SIBLING

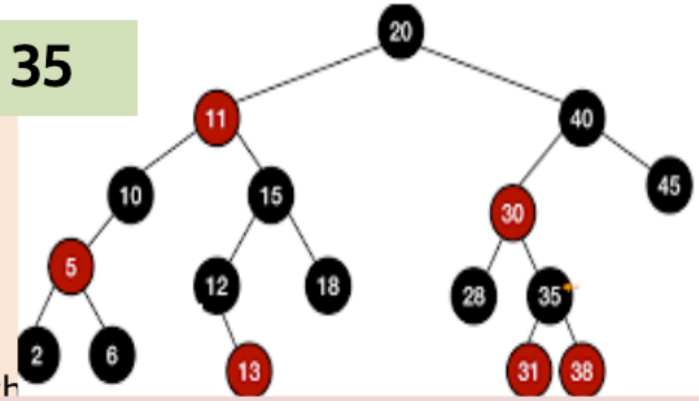
→Rotate PARENT in DB's direction

→Remove DB

→Change color of RED child(far) to BLACK

RB Tree Deletion

Delete 30, 45, 10, 12, 35



Step 1 - Perform BST deletion

Step 2

Case 1-If node to be deleted is **RED** => just delete it [**Terminal**]

Case 2-If node to be deleted is **BLACK** but has **RED** child

=> Replace node with child and color it with BLACK

Or if the Node to be deleted is **BLACK** & has **RED** parent => swap th

Case 3(2) - If **DB** sibling is **RED** (also has **BLACK** parent)

➜ swap colors of PARENT and its SIBLING

➜ rotate PARENT in DB direction

➜ reapply cases

Case 3(3) - If **DB's** sibling is **BLACK** and both of its childr are **BLACK**

➜Remove DB

= => Add extra black to its parent

=====> if parent is RED, then make it BLACK

======> if parent is BLACK, then make it double BLACK

☒ Make sibling RED

➜ If stillDB exists, apply other cases.

Case 3(4) - DB's sibling is **BLACK**, check sibling child's who is far from DB is **BLACK** but near child to DB is **RED**

➜Swap color of DB's sibling & siblings child who is near to DB

➜Rotate sibling in opposite direction to DB

➜Apply case 6

Case 3(5) - DB's sibling is **BLACK** and far child is **RED** [**Terminal**]

➜Swap color of PARENT & SIBLING

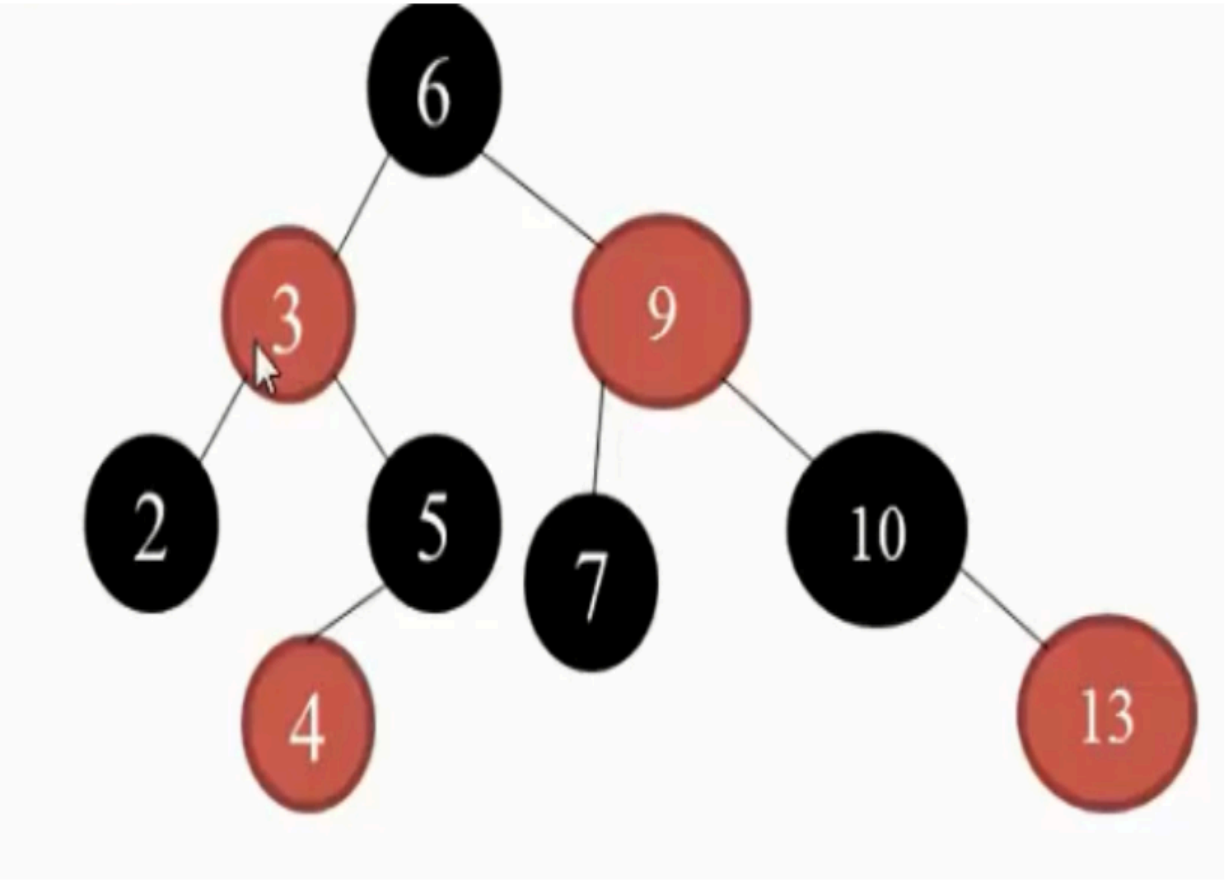
➜Rotate PARENT in DB's direction

➜Remove DB

➜Change color of RED child(far) to BLACK

[if node to be deleted is black , it will be replaced by Double Black DB]

Delete **9, 3, 4, 10, 13, 2**



Step 1 - Perform BST deletion

Step 2

Case 1-If node to be deleted is **RED** => just delete it [**Terminal**]

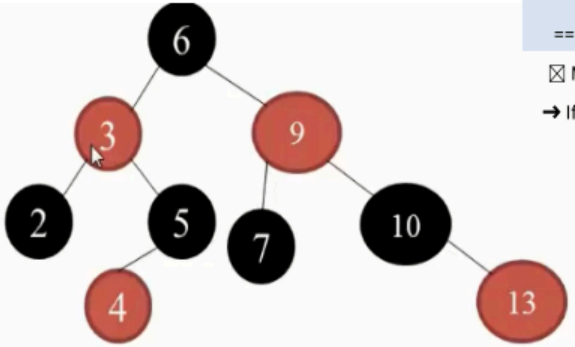
Case 2-If node to be deleted is **BLACK** but has **RED** child

=> Replace node with child and color it with BLACK

Or if the Node to be deleted is **BLACK** & has **RED** parent => swap the color of Parent & Sibling (terminal)

RB Tree Deletion

Delete 9, 3, 4, 10, 13, 2



Case 3(1) If root node is DB, => remove DB [**Terminal**]

Case 3(2) - If **DB** sibling is **RED** (also has **BLACK** parent)

→ swap colors of PARENT and its SIBLING

→ rotate PARENT in DB direction

→ reapply cases

Case 3(3) - If **DB**'s sibling is **BLACK** and both of its children are **BLACK**

→Remove DB

= => Add extra black to its parent

=====> if parent is RED, then make it BLACK

======> if parent is BLACK, then make it double BLACK

☒ Make sibling RED

→ If stillDB exists, apply other cases.

Case 3(4) - DB's sibling is **BLACK**, check sibling child's who is far from DB is **BLACK**

but near child to DB is **RED**

→Swap color of DB's sibling & siblings child who is near to DB

→Rotate sibling in opposite direction to DB

→Apply case 3(5)

Case 3(5) - DB's sibling is **BLACK** and far child is **RED** [**Terminal**]

→Swap color of PARENT & SIBLING

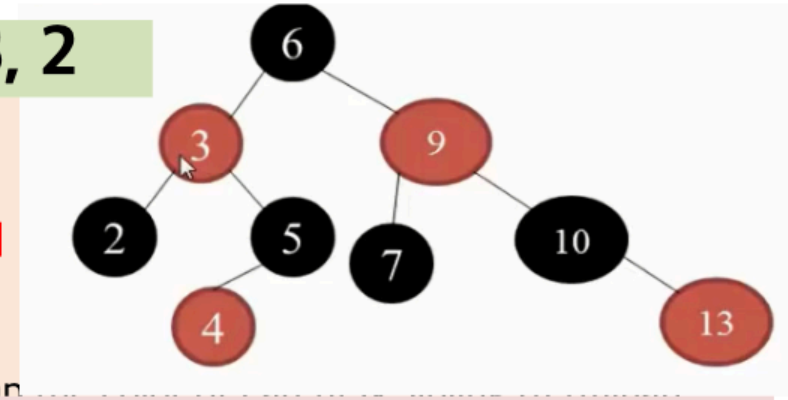
→Rotate PARENT in DB's direction

→Remove DB

→Change color of RED child(far) to BLACK

RB Tree Deletion

Delete 9, 3, 4, 10, 13, 2



Step 1 - Perform BST deletion

Step 2

Case 1-If node to be deleted is **RED** => just delete it [**Terminal**]

Case 2-If node to be deleted is **BLACK** but has **RED** child
=> Replace node with child and color it with BLACK

Or if the Node to be deleted is **BLACK** & has **RED** parent => swap

Case 3(2) - If **DB** sibling is **RED** (also has **BLACK** parent)

➜ swap colors of PARENT and its SIBLING

➜ rotate PARENT in DB direction

➜ reapply cases

Case 3(3) - If **DB's** sibling is **BLACK** and both of its childr
are **BLACK**

➜Remove DB

= => Add extra black to its parent

=====> if parent is RED, then make it BLACK

======> if parent is BLACK, then make it double BLACK

☒ Make sibling RED

➜ If stillDB exists, apply other cases.

Case 3(4) - DB's sibling is **BLACK**, check sibling child's who is far from DB is **BLACK** but near child to DB is **RED**

➜Swap color of DB's sibling & siblings child who is near to DB

➜Rotate sibling in opposite direction to DB

➜Apply case 6

Case 3(5) - DB's sibling is **BLACK** and far child is **RED** [**Terminal**]

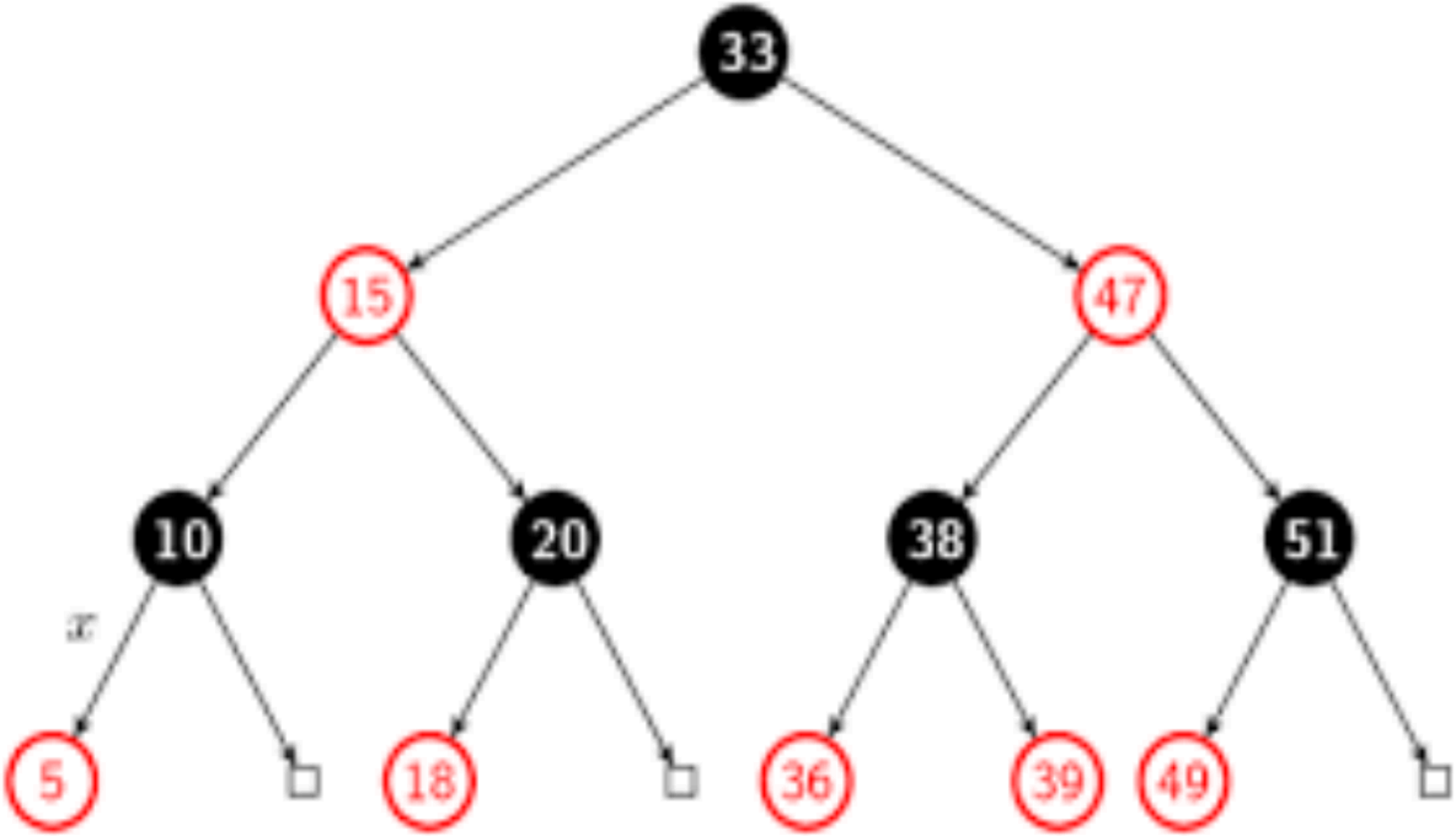
➜Swap color of PARENT & SIBLING

➜Rotate PARENT in DB's direction

➜Remove DB

➜Change color of RED child(far) to BLACK

[if node to be deleted is black , it will be replaced by Double Black DB]


33
15
47
10
20
38
51
x
5
18
36
39
49

Video Lecture of the topic can be found at

How to Delete Node in Binary Search Tree (BST)
https://www.youtube.com/watch?v=ad1KFYHZyMc

Red Black Tree and Insertion Operation
https://www.youtube.com/watch?v=H0MutN9u-Ik

Understanding RB Tree Deletion cases with an Example
https://youtu.be/1qEjtfR0gIo

RB Tree Deletion Example No.2
https://youtu.be/MjlQo2vRAnI

RB Tree Deletion Example No.3
https://youtu.be/vVRo_WagRLQ